

A Universal Zero-Witness Forgery against Miraidon-S

Cryptanalysis of *Miraidon: MinRank Identification*

ePrint Signature Audit
AI-assisted cryptanalysis report

7 October 2026

Abstract

We give a public-key-only universal fresh-message forgery against Miraidon-S as specified in *Miraidon: MinRank Identification*, IACR ePrint 2026/997, version 4. The protocol is intended to prove knowledge of a coefficient vector whose public matrix combination has rank exactly r . The verifier, however, accepts a factorization $Q = LR$ without checking that Q has exact rank r , while the paper’s printed homogeneous MinRank problem also omits a nonzero condition on the coefficient vector. A forger sets the purported coefficient vector and all matrix responses to zero. Both verifier branches then hash the same zero matrix for every first challenge, eliminating the fixed-weight Fiat–Shamir challenge-guessing cost. The attack uses no secret information, signing query, MinRank computation, or random-oracle grinding, and succeeds for every public key and message.

We reproduce the complete Fiat–Shamir forgery at the Level-I, Level-III, and Level-V parameter rows. A full Level-I run using an exact Algorithm-5 key took 0.49 seconds for the forgery and 4.94 seconds including key generation, honest signing, and controls. A separate implementation reproduced two fresh forgeries at every parameter row. An exact-rank check rejects every forgery and preserves honest signatures. The result was AI-discovered and independently AI-reproduced; independent human verification is pending. No originality claim is made for the general technique of detecting a missing nondegeneracy or relation-membership check.

Result status. Practical full-parameter design attack: universal fresh-message forgery with zero signing queries. Discovery, cryptanalysis, implementation, adversarial review, and independent reproduction were AI-assisted. Independent human verification is pending.

1 Introduction

Miraidon-S is a Fiat–Shamir signature derived from a five-pass MinRank identification protocol. Its target paper, by Cartor and Slaughter, is titled *Miraidon: MinRank Identification* [1]. The paper proposes parameter sets for NIST security Levels I, III, and V and analyzes both generic MinRank attacks and fixed-weight attacks on repeated five-pass protocols. The attack in this paper bypasses both analyses: a malformed rank-zero response is accepted by both protocol branches, so the forger need not predict either challenge.

Our contributions are as follows.

- We identify the mismatch between the exact-rank private relation and the verifier’s at-most- r factorization check. The printed homogeneous MinRank problem compounds the issue by allowing the zero coefficient vector.
- We derive an all-zero transcript that answers both second challenges for every nonzero first challenge. It yields a universal public-key-only forgery after the same seed-tree, Merkle-tree, and Fiat–Shamir processing used by an honest signature.
- We validate the attack on all three published parameter rows using independent implementations, an exact Algorithm-5 Level-I key, honest and changed-message controls, an exact-rank-hardened verifier, and the authors’ pinned interactive proof-of-concept.
- We give a concrete repair: check $\text{rank}(Q) = r$ in every opened branch-zero response (or equivalently require both factors to have rank r), state a nondegenerate witness relation, correct the aggregate checks in Algorithm 6, and revise the soundness proof.

Attack-history statement. We located no previous public attack against exact Miraidon-S in searches as of 7 October 2026. The search covered the exact title, scheme name, ePrint identifier, authors, repository records, and terms concerning zero witnesses, rank zero, missing rank checks, soundness, and forgery. This negative result has moderate confidence: the target is recent, indexes may be incomplete, and unpublished or unindexed observations cannot be excluded. We therefore make no absolute priority claim.

Technique attribution. Generic algebraic and combinatorial MinRank algorithms are well established; examples include support-minors and related algebraic methods [2, 3]. Challenge-guessing attacks on Fiat–Shamir signatures from five-pass identification schemes were developed by Kales and Zaverucha [4] and refined for fixed-weight repetition by Battagliola et al. [5, 6]. Miraidon’s claimed parameters account for those costs. Our attack uses neither class of method. Its essential observation is a missing nondegeneracy check, a standard protocol-review issue for which we claim no technique novelty.

2 Target, notation, and claimed security

2.1 Exact target

The target is Miraidon-S in IACR ePrint 2026/997, version 4, dated 2 June 2026 [1]. The audited PDF has SHA-256 digest

88f0bcbfb4f4a4e7079236d48369d5d692d0c39a89447b2bbe3d7bcd4dcfe21f4.

The accompanying proof-of-concept at <https://github.com/FreemanSlaughter/Miraidon> is pinned to Git commit

289beaa76a75e8500faa6fa75e9c76796fe9f633.

The pinned `Miraidon_ZKP.py` has SHA-256 digest

0523b0b89d3be7cca86b23b933e18a368a211e2e89a8efe2dec86cda3647ae15.

We analyze the ordinary signature Miraidon-S, specifically Algorithms 3–6. We do not claim an attack on the separate ring-signature or linkable-ring-signature security properties.

Let q be a prime, let $M_1, \dots, M_k \in \mathbb{F}_q^{m \times n}$ be the public matrices, and let r be the target rank. For $z \in \mathbb{F}_q^k$, write

$$M(z) = \sum_{j=1}^k z_j M_j.$$

Problem 1 in the target asks for $a \in \mathbb{F}_q^k$ such that $\text{rank}(M(a)) \leq r$. It does not state $a \neq \mathbf{0}$. Thus its literal homogeneous formulation is satisfied by $a = \mathbf{0}$ for every instance. Algorithms 3 and 4 instead describe the private witness as an a for which $E = M(a)$ has rank exactly r . Algorithm 5 generates a nonzero honest secret and rejects unless the resulting relation has rank r ; those key-generation properties do not constrain a malicious signature response.

2.2 One protocol round

For an honest round, the prover samples $u \in \mathbb{F}_q^k$ and $S \in \text{GL}_m(\mathbb{F}_q)$, $T \in \text{GL}_n(\mathbb{F}_q)$, then computes

$$X = \sum_{j=1}^k u_j S M_j T = S M(u) T,$$

$$Q = S E T = S M(a) T,$$

$$Q = L R,$$

where $L \in \mathbb{F}_q^{m \times r}$ and $R \in \mathbb{F}_q^{r \times n}$. It commits to $c_0 = \text{Com}(X, Q)$ and $c_1 = \text{Com}(S, T)$. After a first challenge $\xi \in \mathbb{F}_q^*$, it sets

$$v = u + \xi a, \quad y = \text{Com}\left(\sum_{j=1}^k v_j S M_j T\right).$$

The verifier then gives a bit challenge b :

- if $b = 0$, the prover opens X, L, R ; the verifier forms $Q = LR$ and checks c_0 and $y = \text{Com}(X + \xi Q)$;
- if $b = 1$, the prover opens a seed for S, T and the vector v ; the verifier checks c_1 and

$$y = \text{Com}\left(\sum_j v_j S M_j T\right).$$

Miraidon-S repeats this protocol t times. The first Fiat–Shamir hash FS_1 produces t nonzero field elements. The second hash FS_2 produces a bit vector of fixed weight w ; its support I selects the $b = 1$ rounds. A seed tree compresses the disclosed seeds, and a Merkle tree authenticates the branch-zero commitments.

2.3 Claimed concrete security

The target states that Miraidon-S is secure against forgery attacks based on the hardness of MinRank and selects parameters for NIST Levels I, III, and V. Table 1 records the exact suggested rows used in our tests. The final column is the target’s claimed work exponent for its analyzed fixed-weight forgery attack. The target also reports minimum generic MinRank estimates of approximately 2^{136} , 2^{200} , and 2^{265} operations for these rows. These estimates do not apply to the verifier-acceptance flaw.

Table 1: Published Miraidon-S rows and claimed fixed-weight forgery cost.

Level	q	m	n	k	r	t	w	$\log_2 \text{cost}$
I	127	10	10	44	4	180	121	128.29
III	257	12	12	45	6	317	248	192.05
V	509	13	13	50	7	512	435	256.14

2.4 The Algorithm-6 aggregate-check mismatch

Literal Algorithm 6 is inconsistent with signing. Algorithm 4 makes c_0 the root of a Merkle tree over the per-round $c_{0,i}$ values and makes c_1 a flat commitment to all $c_{1,i}$. The printed verifier applies a flat commitment to the $c_{0,i}$ values and applies the Merkle verification to the $c_{1,i}$ values. The declared signature contains a proof from the c_0 tree, so the literal operations do not verify an honestly generated signature.

Our analysis and implementations use the evident consistency correction required by the signing algorithm and the surrounding optimization text:

$$c_0 = \text{VerifyTree}(\{c_{0,i}\}_{i \notin I}, \text{Proof}), \quad c_1 = \text{Com}(c_{1,1}, \dots, c_{1,t}). \quad (1)$$

Honest controls pass this corrected verifier. The attack below passes the same verifier. It is therefore independent of the aggregate-check transcription error, but the scope of our conclusion is necessarily the intended, executable verifier rather than literal Algorithm 6.

3 The zero-witness forgery

3.1 The missing exact-rank constraint

The factor dimensions in an opened branch-zero response imply only

$$\text{rank}(LR) \leq r.$$

They do not imply equality, and Algorithm 3 or 6 performs no rank check on L , R , or $Q = LR$. This distinction is load-bearing. The target’s soundness discussion says that Q has rank r “by construction,” but that statement holds only for an honest prover. In contrast, the completeness proof itself describes Q as having rank at most r . Verification never enforces the exact-rank premise used by extraction.

The most direct malicious choice is

$$a^* = \mathbf{0}, \quad E^* = Q = L = R = \mathbf{0}. \quad (2)$$

For any $u \in \mathbb{F}_q^k$, any valid invertible S, T , and any $\xi \in \mathbb{F}_q^*$, set

$$v = u, \quad X = \sum_{j=1}^k u_j S M_j T.$$

Then the two verifier branches compute the same matrix:

$$X + \xi Q = X = \sum_{j=1}^k v_j S M_j T. \quad (3)$$

The specialization $u = v = \mathbf{0}$ also makes $X = \mathbf{0}$. It removes even the public-matrix arithmetic: the attacker need not read the matrices M_j .

Theorem 1 (Universal accepting transcript). *For every Miraidon-S public key, every message, every vector of nonzero first challenges, and every fixed-weight second challenge, the choices in Equation (2) with $u = v = \mathbf{0}$ produce responses accepted by the intended verifier in every round.*

Proof. Before either challenge, the attacker can commit to $X = Q = \mathbf{0}$ and to honestly expanded invertible S, T . For $b = 0$, the verifier reconstructs $Q = LR = \mathbf{0}$ and hashes $X + \xi Q = \mathbf{0}$. For $b = 1$, it reconstructs the committed S, T and hashes $\sum_j v_j S M_j T = \mathbf{0}$. Hence the same delayed commitment $y = \text{Com}(\mathbf{0})$ is correct for either bit and for every ξ . The attacker knows all commitment openings, seeds, and authentication paths, so the corrected aggregate checks in Equation (1) also pass. This holds independently in all t rounds. $\square$

3.2 Forgery algorithm

The following algorithm spells out the complete noninteractive forgery. It uses the target's abstract seed-tree, Merkle-tree, commitment, and Fiat–Shamir interfaces; concrete encodings are discussed in Section 7.

Algorithm: universal zero-witness forgery.

Input: a public key pk , parameter row (q, m, n, k, r, t, w) , and an attacker-chosen message msg .

1. Sample the master seed and salt and derive the t per-round seeds as an honest signer would.
2. For each $i \in [t]$, expand the seed to valid $S_i \in \text{GL}_m(\mathbb{F}_q)$ and $T_i \in \text{GL}_n(\mathbb{F}_q)$, and set

$$u_i = v_i = \mathbf{0}, \quad X_i = Q_i = L_i = R_i = \mathbf{0}.$$

Compute $c_{0,i} = \text{Com}(X_i, Q_i, \text{Salt}, i)$ and $c_{1,i} = \text{Com}(S_i, T_i, \text{Salt}, i)$.

3. Build the c_0 Merkle tree, let c_0 be its root, and set $c_1 = \text{Com}(c_{1,1}, \dots, c_{1,t})$.
4. Query $(\xi_1, \dots, \xi_t) = \text{FS}_1(c_0, c_1, \text{msg}, \text{Salt})$. For every round set $y_i = \text{Com}(\mathbf{0})$ and $Y = \text{Com}(y_1, \dots, y_t)$.
5. Query $b = \text{FS}_2(c_0, c_1, \text{msg}, \text{Salt}, (\xi_i)_{i \in [t]}, Y)$ and let $I = \text{supp}(b)$.
6. Form the honest seed opening for I and the honest Merkle multiproof for the opened $c_{0,i}$ values. For $i \notin I$, output $(c_{1,i}, X_i, L_i, R_i)$. For $i \in I$, output v_i ; the seed path reveals the corresponding S_i, T_i .
7. Return the resulting signature $(c_0, c_1, \text{Salt}, Y, \text{Path}, \text{Proof}, \{\text{resp}_i\}_{i \in [t]})$.

The attacker never calls a signing oracle and never evaluates a secret-key operation. Because every response is prepared honestly relative to the malicious zero statement, there is no need to grind either Fiat–Shamir hash. Binding of the commitment and collision resistance of the Merkle tree do not help: the attacker opens exactly what it committed.

4 Complexity and success probability

The attack performs one ordinary transcript-construction pass. In the all-zero specialization it avoids the k public-matrix linear combination and the rank- r factorization in every round. Its remaining work consists of seed expansion, generation of invertible masking matrices, hashing, tree construction, and serialization. With the dimensions treated explicitly, this is polynomial in the parameter sizes and linear

in the number of rounds apart from the cost of producing S_i, T_i . Its signature size is the honest signature size for the same row.

The success probability is 1 for every well-formed output of the specified Fiat–Shamir functions. In particular, it is independent of q , t , and w . This contrasts with the target’s $2^{128.29}$, $2^{192.05}$, and $2^{256.14}$ estimates for challenge-guessing forgeries. The attack also does not solve a MinRank instance, so generic MinRank cost estimates are irrelevant.

Every published row is affected. The factor matrices with the required shapes exist as zero matrices for $r = 4, 6, 7$, and both second-challenge branches occur in the fixed-weight signatures. The exact branch counts observed in a deterministic reproduction are given in Table 2.

Table 2: Full-scale all-row reproduction. “Fresh” counts separately generated attacker-chosen messages.

Level	$b = 0$	$b = 1$	time (s)	fresh 1	fresh 2	changed	rank fix	honest
I	59	121	2.684	pass	pass	reject	reject	pass
III	69	248	7.329	pass	pass	reject	reject	pass
V	77	435	17.256	pass	pass	reject	reject	pass

Here “rank fix” is the exact-rank-hardened verifier applied to the forgery; “honest” is an honest signature accepted by that hardened verifier. All first challenges were nonzero. Every opened forged Q had rank zero, while the honest controls opened matrices of ranks 4, 6, and 7 respectively.

5 Experimental validation

5.1 Primary exact-key experiment

The primary dependency-free Python implementation instantiated the natural Algorithm-5 generator at the exact Level-I row (127, 10, 10, 44, 4, 180, 121). With deterministic test randomness, key generation solved the 44-equation/43-ratio system after 199 attempts, reconstructed the specified 16-element compressed public-key tail, and confirmed that the honest secret combination had rank 4.

On that key, the same corrected verifier accepted an honest signature and a forgery on a distinct attacker-chosen message. The attacker function received only the public matrices, the fresh message, and attacker randomness. It did not receive the secret key and made zero signing queries. Replaying the signature on a changed message failed. Adding an exact-rank test rejected the forgery and continued to accept the honest signature.

The forge phase took 0.490 seconds. The complete key-generation, honest-signing, forging, verification, and control run took 4.94 seconds and 22,528 KiB peak resident memory on one worker. The recorded environment used Python 3.12.3 on an AMD EPYC processor under KVM. This experiment made one forgery attempt and obtained one acceptance; the algebraic proof above gives the success statement beyond that single timing trial.

5.2 Independent implementations

A fresh-context standard-library implementation independently reconstructed the complete signature system: finite-field and matrix arithmetic, valid target-shaped keys, seed expansion, compressed seed disclosure, Merkle multiproofs, both Fiat–Shamir stages, an honest signer, the attacker, and the corrected verifier. It generated two accepted fresh forgeries at each of the three rows, for an observed 6/6 full-signature acceptance rate. All changed-message replays failed; all exact-rank controls rejected the forgeries and accepted the honest signatures. The all-row run took 27.38 seconds and 35,328 KiB peak resident memory on one worker.

A second independent Level-I implementation serialized a public key, an honest signature, two fresh zero-witness forgeries, a nonzero- v variant, and a nonzero-factor variant. It exercised message, response, Merkle-proof, seed-opening, and Y tampering controls; each malformed signature was rejected. Its main run took approximately 5.2 seconds and 24 MiB peak memory.

Finally, a branch probe imported the pinned authors’ interactive proof-of-concept rather than reimplementing its verifier. At the exact Level-I dimensions, the zero transcript passed both $b = 0$ and $b = 1$ for all 126 values of $\xi \in \mathbb{F}_{127}^*$: 252 accepted branch cases out of 252. Changing an opened X or v while

retaining the commitment caused rejection. This confirms that acceptance follows from the protocol equation rather than from disabled commitment checks.

5.3 Controls and repair boundary

The independent controls delimit what must be checked.

- Requiring $v \neq \mathbf{0}$ is insufficient. Set $u = v = e_1$, $Q = 0$, and $X = SM_1T$; Equation (3) still holds.
- Requiring merely $L \neq 0$ and $R \neq 0$ is insufficient. For the published $r \geq 2$, nonzero factors may be supported on different latent coordinates so that $LR = 0$. This variant was accepted.
- Requiring full-rank S and T is insufficient; the target already does so, and all experiments used invertible masks.
- Checking $\text{rank}(Q) = r$ rejects the demonstrated family and preserves honest signatures in every tested row.

6 Repair

The direct verifier repair is to reject every opened branch-zero response unless

$$\text{rank}(Q) = \text{rank}(LR) = r. \quad (4)$$

Equivalently, the verifier may require $L \in \mathbb{F}_q^{m \times r}$ to have full column rank and $R \in \mathbb{F}_q^{r \times n}$ to have full row rank. These conditions force $\text{rank}(LR) = r$ by Sylvester’s rank inequality. Checking $Q \neq 0$ alone blocks this particular zero- Q family, but it does not enforce the exact-rank relation stated by Algorithms 3 and 4.

A complete specification repair should also:

1. define the homogeneous MinRank witness as nonzero or projectively normalized and state unambiguously whether the relation is exact rank r ;
2. add Equation (4), or the equivalent two factor-rank checks, to Algorithms 3 and 6;
3. correct the swapped c_0 Merkle and c_1 flat aggregate checks as in Equation (1); and
4. revise the soundness argument so that exact rank is established by a verifier check rather than asserted “by construction.”

The repaired construction requires a fresh proof review; our experiments show only that the exact-rank check blocks this attack while preserving the tested honest executions.

7 Reproducibility and limitations

The public reproducer uses only Python’s standard library. From the root of the public audit repository, run

```
python3 attacks/miraidon-s/reproduce.py \
--output attacks/miraidon-s/result.json
```

It generates an exact Algorithm-5 Level-I key, checks the honest relation, signs and verifies an honest message, forges a distinct fresh message without passing secret material to the attacker, rejects a changed-message replay, and exercises the exact-rank repair. The checked-in `attacks/miraidon-s/reference-output.json` records deterministic public key, message, and signature digests; timing fields are machine-dependent.

The target paper specifies the signature at the algorithm level but does not provide a complete Sign/Verify implementation, byte encoding, concrete Fiat–Shamir expansion, or tree encoding. The reproductions therefore use unambiguous, domain-separated SHAKE256-based reference encodings and ordinary binary tree constructions. This is a limitation for byte-for-byte interoperability, not for the algebraic result: both branches receive the same matrix before hashing, and the attacker knows every tree and commitment opening. Two implementations with different framing and tree code reached the same result, and the interactive branch identity passed in the pinned source.

The experimental timings are reproducibility measurements rather than optimized benchmarks. The exact Algorithm-5 run used one deterministic key and one timed forgery; the independent implementation covered two fresh messages per parameter row. The universal success claim follows from Equation (3),

not statistical extrapolation. This paper analyzes version 4 and the intended aggregate-check correction described in Section 2.4; a later specification may differ. No secret-key recovery is claimed because the stronger direct-forgery outcome already breaks message authenticity.

8 AI and verification disclosure

This report is an AI-assisted research artifact. The participation record is:

Activity	Status
Target discovery and triage	AI-assisted
Attack-history search	AI-run with cached public-source evidence
Cryptanalysis	Codex and specialist AI agents
Implementation and experiments	AI-generated and AI-run
Adversarial review	Separate AI agent
Independent reproduction	Fresh-context AI agents and independent code
Independent human verification	Pending

AI reproduction is not human verification. The result should be cited and reviewed with that distinction intact.

9 Conclusion

Miraidon-S version 4 accepts a rank-zero factorization in a protocol whose soundness argument requires exact rank r . The all-zero coefficient vector therefore gives the forger one prechallenge transcript that answers every first challenge and both second-challenge branches. Parallel repetition, fixed-weight challenges, Fiat-Shamir hashing, and tree compression preserve rather than remove this identity. The resulting attack is a practical, public-key-only fresh-message forgery against every published parameter row. An exact-rank verifier check blocks the demonstrated forgery, but the relation, verification algorithm, and soundness proof should be repaired together.

References

- [1] R. Cartor and F. Slaughter. Miraidon: MinRank Identification. Cryptology ePrint Archive, Paper 2026/997, version 4, 2026. <https://eprint.iacr.org/2026/997>.
- [2] M. Bardet, M. Bros, D. Cabarcas, P. Gaborit, R. Perlmeyer, D. Smith-Tone, J.-P. Tillich, and J. Verbel. Improvements of algebraic attacks for solving the rank decoding and MinRank problems. In *ASIACRYPT 2020*, pages 507–536. Springer, 2020.
- [3] M. Bardet, P. Briaud, M. Bros, P. Gaborit, and J.-P. Tillich. Revisiting algebraic attacks on MinRank and on the rank decoding problem. *Designs, Codes and Cryptography*, 91(11):3671–3707, 2023. <https://doi.org/10.1007/s10623-023-01265-x>.
- [4] D. Kales and G. Zaverucha. An attack on some signature schemes constructed from five-pass identification schemes. In *CANS 2020*, pages 3–22. Springer, 2020. https://doi.org/10.1007/978-3-030-65411-5_1.
- [5] M. Battagliola, R. Longo, F. Pintore, E. Signorini, and G. Tognolini. A revision of CROSS security: Proofs and attacks for multi-round Fiat-Shamir signatures. Cryptology ePrint Archive, Paper 2025/127, 2025. <https://eprint.iacr.org/2025/127>.
- [6] M. Battagliola, R. Longo, F. Pintore, E. Signorini, and G. Tognolini. Security of fixed-weight repetitions of special-sound multi-round interactive proofs. *Designs, Codes and Cryptography*, 93(10):4323–4353, 2025. <https://doi.org/10.1007/s10623-025-01686-w>.
- [7] N. T. Courtois. Efficient zero-knowledge authentication based on a linear algebra problem MinRank. In *ASIACRYPT 2001*, pages 402–421. Springer, 2001.

- [8] A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO '86*, pages 186–194. Springer, 1987.
- [9] A. J. De Scala and C. Sanna. Smaller public keys for MinRank-based schemes. *Journal of Mathematical Cryptology*, 19(1):20240008, 2025. <https://doi.org/10.1515/jmc-2024-0008>.